



DIPLOMADO

Desarrollo de sistemas con tecnología Java

Módulo 6

Introducción de Aplicaciones Empresariales
con Spring Framework

Mtro. ISC Miguel Ángel Sánchez Hernández

Objetivo

Aprender los diferentes tipos de configuración basadas en java que nos ofrece Spring para un Bean.



Lo que veremos

- Configuraciones basadas en Java
- La vida de un Bean



Anotaciones de Spring

Se analizo como hacer las configuraciones de un bean en XML, ahora lo haremos con configuración basada en Java (anotaciones). Estas son las anotaciones agrupadas por categoría

Ciclo de Vida y Configuración: Poder controlar el inicio y destrucción de los beans

@PostConstruct	El método que se ejecutara después de el bean se ha iniciado.
@PreDestroy	Método que se ejecuta antes que el bean sea destruido.
@Bean	Crear un bean en una clase de configuración.
@Configuration	Clase que sirve como fuente para crear beans.
@ComponentScan	Escanear paquetes para detectar componentes para Spring

Anotaciones de Spring

Anotaciones de Componentes: las clases las interpreta Spring para ser beans

@Component	La clase se interpreta como componente genérico de Spring.
@Service	Clases que tendrán lógica de negocio y son más especializadas que @Component.
@Repository	Para clases de acceso de datos DAO, traduce las excepciones específicas a excepciones de Spring.
@Controller	Clases para controladores web para aplicaciones Spring MVC.
@RestController	Junta @ResponseBody y @Controller, para un API REST para clases.

Anotaciones de Spring

Inyección de Dependencias: Permitir que Spring inyecte dependencias automáticamente

@Autowired	Inyecta dependencia automáticamente por tipo.
@Qualifier("nombre")	Se puede especificar cual bean se inyecta cuando hay más de un candidato.
@Inject	Parecido a @Autowired (JSR-330).
@Resource(name="bean")	Inyecta por nombre (JSR-250).
@Value("\${name.valor}")	Desde archivos de propiedades inserta valores

Anotaciones de Spring

Spring MVC: Ocupar Web

@RequestMapping	Maapeo de una URL HTTP a un método de un controlador.
@GetMapping @PostMapping @DeleteMapping @PutMapping	Abreviación de @RequestMapping(method="RequestMethod.GET"). @RequestMapping(method="RequestMethod.POST"). @RequestMapping(method="RequestMethod.DELETE"). @RequestMapping(method="PUT").
@PathVariable	Vinculación de variables URL a parámetros de un método.
@RequestParam	Obtener los de la consulta (?parametro1=valor,parametro2=valor,etc).
@RequestBody	El valor a devolver debe ser como respuesta HTTP (JSON, XML, etc).
@ModelAttribute	Hacer el vinculo de los datos del modelo a un formulario o un objeto.
@ExceptionHandler	Control de excepciones especificas del controlador.

Anotaciones de Spring

Spring Security: Seguridad en Spring

@Secured("ROLE_ADMIN")	Poder restringir a ciertos métodos.
@PreAuthorize("hasRole('ADMIN')")	Expresiones SpEL que controlan el acceso.
@EnableWebSecurity	Poder habilitar la configuración de seguridad en la web.

Anotaciones de Spring

Anotaciones útiles

@SpringBootApplication	Junta las anotaciones @Configuration, @EnableAutoConfiguration y @ComponentScan.
@EnableAutoConfiguration	Activa la configuración automática de Spring Boot
@Transactional	Método que debe ejecutar dentro de una transacción.

Configuración del Bean basada en Java

Ocuparemos ahora anotaciones como `@Bean`, anotadas debajo de métodos y todos dentro de una clase con la anotación `@Configuration`. Todas estas definiciones de los beans serán una relación directa a los objetos reales que componen nuestra aplicación. Pueden ser objetos para una capa de servicio, acceso a datos (DAO) etc.

Objetivos de la práctica siguiente

Los objetivos de la practica son:

- Aprender como indicar al contenedor de Spring que registre un bean por configuración basada en Java.
- Como resolver el problema de tener beans de alcance singleton.
- Combinar configuraciones basada en Java con configuraciones XML
- Indicarle a Spring que descubra los beans que se crean en el contexto de aplicación.



Ejercicio 12: Configuración basada en Java (@Configuration,@Bean)

Objetivos de la práctica siguiente

Los objetivos de la practica son:

- Crear una clase POJO con @Component para obtener beans DAO.
- Ocupar @Autowired para brindar un servicio con DAO.
- @Autowired en un método con el atributo required.
- @Autowired en el constructor.
- Resolviendo ambigüedad en @Autowired con @Primary y @Qualifier
- @Qualifier por constructor, propiedad y método.



Ejercicio 13: POJO, @Autowired

Contacto

Mtro. ISC Miguel Ángel Sánchez Hernández

miguelsanchezt32@aragon.unam.mx